

DEPARTMENT OF COMPUTER SCIENCE
FALL 2025

CS4110 - High Performance
Computing with GPUs

Deliverable 1
Report

By:
Fatima Sohail [23i-0633]
Hareem Zia [23f-0842]
HPC-A

Github Link:

<https://github.com/FatimaSohail/KLT-Feature-Tracker-Acceleration-GPUs>

Profiling Analysis:

The profiling of KLT Feature Tracking Application was conducted using realistic datasets with varying frame counts, feature workloads and resolutions to understand how these parameters influence the computational performance and hotspot distribution and GPU porting decisions.

The computational workload of the KLT tracking application is proportional to: *Workload*
 $= \text{Number of Frames} * \text{Number of Features per Frame} * \text{Operations per Feature}$

The workload in this application is primarily defined by the number of frames processed and the number of features tracked in each frame. As the workload increases, the system must perform significantly more convolution and tracking operations. This increased computational intensity directly affects runtime and resource utilization. Profiling reveals that most of the time is spent in repetitive image-processing functions, making them suitable for GPU acceleration.

We profiled the application on the provided 10 frame dataset and two larger realistic datasets with 550 frames, at resolutions of 480p and 4K respectively. In KLT, each frame undergoes multiple processing steps, including smoothing, gradient computation, and feature tracking. As the number of frames increases, the total number of function calls to computationally heavy processes such as convolution and gradient calculation grows proportionally. Similarly, the number of features determines the number of tracking computations per frame, so a higher feature count increases per-frame computational workload, processing effort and memory usage.

Resolution defines how many pixels each convolution and gradient operation must process. Higher resolution (e.g., 4K) frames contain significantly more pixels than lower-resolution (480p) ones, meaning that even with the same number of frames and features, each frame requires far more computations and memory accesses. As a result, the runtime of convolution and gradient-related functions scales multiplicatively with resolution, increasing both arithmetic intensity and data movement. While this improves tracking precision, it also increases memory use and computation time, making efficient parallelization essential.

The profiling results revealed two major hotspots, `_convolveImageHoriz`, and `_convolveImageVert`. For both 480p and 4K workloads, both functions accounted for over 80% of the total CPU time combined, while their execution count increased linearly with the number of frames. The jump from 480p to 4K further increased their runtime, as each pixel-level convolution involved significantly larger image grids.

The convolution functions show pixel-level independence, making them well-suited for GPU parallelization with little synchronization overhead except at filter boundaries. As each pixel can be processed independently, mapping convolution to thousands of GPU threads can significantly reduce runtime.

CPU and GPU Communication Analysis

The decision to port these functions to GPU was carefully guided by the CPU-GPU communication costs. While profiling identified that the functions `convolveImageVert()` (54.18%) and `convolveImageHoriz` (36.57%) dominate the runtime, GPU acceleration is only beneficial when the computation-to-communication ratio is sufficiently high. Since these functions perform a major part of the computation, the communication overhead can be considered small compared to the massive speedup of processing thousands of pixels per frame in parallel.

On the other hand, `trackFeature()` (2.7%) contributes less to the total runtime but internally relies on computationally heavy subroutines, `computeIntensityDifference()` (1.23%) and `computeGradientSum()` (1.16%) which is not computationally heavy on its own but becomes significant due to the extremely high call frequency of `interpolate()` (~47 million times). Porting `interpolate()` alone to the GPU alone would result in incur excessive communication and kernel launch overhead (~47 million kernel launches), neglecting any potential speedup.

Instead, by porting `computeIntensityDifference()` and `computeGradientSum()` to the GPU as kernel functions along with `interpolate()` as a device function, the communication overhead is drastically reduced. In this configuration, the number of kernel launches is reduced from ~47 million to approximately ~353 thousand. Moreover, porting `trackFeatures()` to the GPU will reduce the number of kernel launches from two to one, significantly reducing the launch overhead. This design takes advantage of the parallel throughput GPU provides while ensuring that the communication/launch overhead does not dominate the speedup.

Lastly, the PPM read/write functions perform I/O operations that interact directly with the file system. Since the GPU cannot perform file I/O operations, these functions must remain on the CPU, ensuring that disk access and data management occur efficiently.

Final Decision

The functions **`convolveImageVert()`**, **`convolveImageHoriz()`**, **`computeIntensityDifference()`**, **`computeGradientSum()`**, **`interpolate()`**, and **`trackFeatures()`** will be ported to the GPU. However only `convolveImageVert()` and `convolveImageHoriz()` will be ported in V2 (naive implementation) while the rest will be ported in V3 (optimised implementation).

Conclusion

By considering both profiling results and communication overhead, the true computational hotspots to be ported are decided. These will cause minimum unnecessary data transfers, reduce launch overhead, and ensure efficient utilisation of both CPU and GPU resources. This will result in a balance between parallel throughput and computation efficiency, delivering overall speedup without excessive synchronization issues.